

The Process Model Domain-Specific Language

To represent process models, we introduce a formal Domain-Specific Language (DSL). The DSL uses three finite, pairwise disjoint sets: node identifiers (E), guard variables (V), and actor identifiers (A). Its predicates describe process nodes, variables, actors, and actor assignments (S), as well as control flow (P), guards (G), and text labels (L).

Below, we define these predicates. We then specify the grammar of the DSL using Extended Backus–Naur Form (EBNF).

Each node identifier $e \in E$ has exactly one node type: *init*, *end*, *action*, *branch*, or *converge*. The following predicates define the process model.

1. **init**(e): Defines the starting point of the process.
2. **end**(e): Defines a terminal point of the process.
3. **action**(e, str): Defines an executable step or task, where $str \in L$ is its natural-language description.
4. **branch**(e, t): Splits the control flow according to the branching type t :
 - *AND*: Activates all outgoing flows in parallel.
 - *XOR*: Activates exactly one outgoing flow based on its condition.
 - *OR*: Activates one or more outgoing flows based on their conditions.

In this manuscript, we also use the compact form **branch**(e, t, str) to represent an actionable decision point. It represents the same behavior as

$$\text{action}(a, str), \quad \text{branch}(b, t), \quad \text{precedence}(a, b),$$

where $a, b \in E$ are distinct node identifiers. In this expanded form, incoming flows connect to the action node a , and outgoing flows leave the branch node b . The compact form combines these two steps into one node without changing their behavior.

5. **converge**(e): Combines multiple incoming flows into a single outgoing flow. Although this predicate is part of the DSL, convergence nodes are not explicitly represented in the manuscript, as convergence points can be identified by following the process traces.
6. **defines**($v, str, \mathcal{R}_v$): Declares a variable $v \in V$, where $str \in L$ describes the variable and $\mathcal{R}_v$ defines its possible values. This domain may be a finite set or a continuous interval.
7. **actor**($a, str, [e_1, \dots, e_n]$): Declares an actor $a \in A$ with the natural-language name $str \in L$ and assigns it the nodes $e_1, \dots, e_n \in E$. Each assignment adds a pair (a, e_i) to S , indicating that the actor is responsible for the corresponding node.
8. **precedence**(e, s): Defines a directed control flow transition $(e, s) \in P$ from node $e \in E$ to node $s \in E$. This predicate can also represent transitions that form cycles.
9. **guard**(e, s, ϕ): Assigns a condition $\phi \in G$ to the transition from e to s , where e is an *XOR* or *OR* branch. The transition can be taken only if ϕ evaluates to true. Conditions use variables declared through *defines* and must respect their domains $\mathcal{R}_v$. They may take the following forms:

- *Numerical comparisons*: Expressions of the form $x_1 \text{ op } x_2$, where

$$\text{op} \in \{=, \neq, <, \leq, >, \geq\},$$

and x_1 and x_2 are algebraic terms over numerical variables and constants, such as $(x + y) \cdot z \leq k$.

- *Logical expressions*: Boolean variables or conditions combined using $\wedge$ (AND), $\vee$ (OR), and $\neg$ (NOT), such as $x \vee y$.
- *Categorical or string comparisons*: Equality conditions over discrete values or strings, such as $x = \text{'apple'}$, where $\text{'apple'} \in \mathcal{R}_x$.

10. **label**(e, s, str): Optionally assigns a natural-language label $str \in L$ to the transition from e to s for visualization or traceability.

DSL Grammar

We define the grammar of the DSL using Extended Backus–Naur Form (EBNF). In the grammar below, $\langle \text{ID} \rangle$ represents node identifiers in E , $\langle \text{VAR} \rangle$ represents variables in V , $\langle \text{STR} \rangle$ represents natural-language text in L , and $\langle \text{CONDITION} \rangle$ represents Boolean guard expressions in G . We use the regular expression operators $+$, $*$, and $?$ to indicate one or more occurrences, zero or more occurrences, and an optional occurrence, respectively. Braces are treated as literal terminal symbols.

```
 $\langle \text{PROCESS} \rangle ::= \langle \text{DECL\_BLOCK} \rangle \langle \text{REL\_BLOCK} \rangle$   
 $\langle \text{DECL\_BLOCK} \rangle ::= ( \langle \text{NODE\_DECL} \rangle \mid \langle \text{VAR\_DECL} \rangle \mid \langle \text{ACTOR\_DECL} \rangle ) +$   
 $\langle \text{NODE\_DECL} \rangle ::= ( \text{init} \mid \text{end} \mid \text{converge} ) ( \langle \text{ID} \rangle ) \mid \text{branch} ( \langle \text{ID} \rangle , ( \text{AND} \mid \text{OR} \mid \text{XOR} ) ) \mid \text{action} ( \langle \text{ID} \rangle , \langle \text{STR} \rangle )$   
 $\langle \text{VAR\_DECL} \rangle ::= \text{defines} ( \langle \text{VAR} \rangle , \langle \text{STR} \rangle , \langle \text{RANGE} \rangle )$   
 $\langle \text{ACTOR\_DECL} \rangle ::= \text{actor} ( \langle \text{ID} \rangle , \langle \text{STR} \rangle , [ \langle \text{ID} \rangle ( , \langle \text{ID} \rangle ) * ] )$   
 $\langle \text{REL\_BLOCK} \rangle ::= ( \langle \text{PRECEDENCE} \rangle \mid \langle \text{GUARD} \rangle \mid \langle \text{LABEL} \rangle ) +$   
 $\langle \text{PRECEDENCE} \rangle ::= \text{precedence} ( \langle \text{ID} \rangle , \langle \text{ID} \rangle )$   
 $\langle \text{GUARD} \rangle ::= \text{guard} ( \langle \text{ID} \rangle , \langle \text{ID} \rangle , \langle \text{CONDITION} \rangle )$   
 $\langle \text{LABEL} \rangle ::= \text{label} ( \langle \text{ID} \rangle , \langle \text{ID} \rangle , \langle \text{STR} \rangle )$   
 $\langle \text{RANGE} \rangle ::= \{ ( \langle \text{VAL} \rangle ( , \langle \text{VAL} \rangle ) * \mid [ \langle \text{NUM} \rangle ? - \langle \text{NUM} \rangle ? ] ) ? \}$   
 $\langle \text{CONDITION} \rangle ::= \langle \text{B\_TERM} \rangle ( \text{OR} \langle \text{B\_TERM} \rangle ) *$   
 $\langle \text{B\_TERM} \rangle ::= \langle \text{B\_FACTOR} \rangle ( \text{AND} \langle \text{B\_FACTOR} \rangle ) *$   
 $\langle \text{B\_FACTOR} \rangle ::= \text{NOT?} \langle \text{REL\_CONDITION} \rangle$   
 $\langle \text{REL\_CONDITION} \rangle ::= \langle \text{A\_CONDITION} \rangle ( \langle \text{OP} \rangle \langle \text{A\_CONDITION} \rangle ) ?$   
 $\langle \text{A\_CONDITION} \rangle ::= \langle \text{A\_TERM} \rangle ( \langle \text{ADD\_OP} \rangle \langle \text{A\_TERM} \rangle ) *$   
 $\langle \text{A\_TERM} \rangle ::= \langle \text{A\_FACTOR} \rangle ( \langle \text{MUL\_OP} \rangle \langle \text{A\_FACTOR} \rangle ) *$   
 $\langle \text{A\_FACTOR} \rangle ::= ( \langle \text{CONDITION} \rangle ) \mid \langle \text{VAR} \rangle \mid \langle \text{VAL} \rangle$   
 $\langle \text{ID} \rangle ::= \langle \text{LETTER} \rangle ( \langle \text{LETTER} \rangle \mid \langle \text{DIGIT} \rangle \mid \_ ) *$   
 $\langle \text{VAR} \rangle ::= \langle \text{LETTER} \rangle ( \langle \text{LETTER} \rangle \mid \langle \text{DIGIT} \rangle \mid \_ ) *$   
 $\langle \text{STR} \rangle ::= \text{' ' } \langle \text{CHAR} \rangle * \text{' ' } \mid \text{' ' } \langle \text{CHAR} \rangle * \text{' ' }$   
 $\langle \text{CHAR} \rangle ::= \langle \text{LETTER} \rangle \mid \langle \text{DIGIT} \rangle \mid \langle \text{SYM} \rangle$   
 $\langle \text{VAL} \rangle ::= \langle \text{STR} \rangle \mid \langle \text{BOOL} \rangle \mid \langle \text{NUM} \rangle$   
 $\langle \text{BOOL} \rangle ::= \text{true} \mid \text{false}$   
 $\langle \text{NUM} \rangle ::= \text{-?} \langle \text{DIGIT} \rangle + ( \text{.} \langle \text{DIGIT} \rangle + ) ?$   
 $\langle \text{LETTER} \rangle ::= [\text{a-zA-Z}]$   
 $\langle \text{DIGIT} \rangle ::= [0-9]$   
 $\langle \text{SYM} \rangle ::= \text{-} \mid \text{|} \mid \text{,} \mid \text{.} \mid \text{-} \mid \text{!} \mid \text{?} \mid \dots$   
 $\langle \text{OP} \rangle ::= \text{=} \mid \text{!=} \mid \text{<} \mid \text{<=} \mid \text{>} \mid \text{>=}$   
 $\langle \text{ADD\_OP} \rangle ::= \text{+} \mid \text{-}$   
 $\langle \text{MUL\_OP} \rangle ::= \text{*} \mid \text{/}$ 
```